

Programmation Fonctionnelle

TD n° 1 : Premiers pas

1 Mise en place d'un environnement de programmation OCaml

Si vous travaillez sur les machines des salles de TP de Sophie Germain, OCaml est déjà installé : vous pouvez donc passer directement à la Section 2.

1.1 Installation d'OCaml

Pour installer OCaml sur vos ordinateurs personnels, il est recommandé d'utiliser le gestionnaire de paquets [OPAM](#). La commande suivante devrait installer Opam sur macOS et Linux :

```
bash -c "sh <(curl -fsSL https://opam.ocaml.org/install.sh)"
```

Il faut ensuite initialiser opam en tapant :

```
opam init
```

Et sous Windows ? Vous pouvez vous référer aux instructions dans la [documentation OCaml](#). Sinon, utiliser WSL ou une machine virtuelle Linux, et référez-vous aux instructions pour Linux.

1.2 Installation du Toplevel

Le compilateur OCaml (commande `ocamlc`) vous permet de compiler des fichiers OCaml (extension `.ml`) et s'utilise à la manière du compilateur `gcc`. OCaml dispose également d'un interpréteur de commandes (appelé « [Toplevel](#) ») qui permet de tester son code OCaml de façon interactive. C'est ce que nous utiliserons aujourd'hui et dans la plupart des TP. Vous pouvez l'installer avec la commande suivante :

```
opam install utop
```

Ensuite, vous pouvez lancer le toplevel dans un terminal avec la commande `utop`.

1.3 Intégration à un éditeur de texte

Pour plus de confort, il est recommandé d'intégrer OCaml dans un éditeur de texte.

Option 1 (Recommandé) : Emacs

Pour intégrer OCaml dans l'éditeur de texte emacs, il est conseillé d'utiliser les extensions [Merlin](#) et [Tuareg](#). Pour installer emacs, ainsi que ces deux extensions :

```
sudo apt install emacs  
opam install tuareg merlin user-setup
```

Le mode OCaml se lancera dès que vous ouvrirez un fichier `.ml` dans emacs.

Option 2 : VSCode

Pour intégrer OCaml dans VSCode, il faut installer l'extension « OCaml Platform », disponible sur le [Visual Studio Marketplace](#)

Cette extension nécessite d'avoir installé [OCaml-LSP](#). Pour l'installer,

```
opam install ocaml-lsp-server
```

2 Utilisation du Toplevel

Dans un premier temps, nous allons utiliser le Toplevel OCaml. Vous pouvez le lancer dans un terminal avec la commande `utop`.

Exercice 1 : Prévoir le résultat fourni par l'interpréteur Caml après chacune des commandes suivantes, entrées dans l'ordre.

```
#let x = "hello";;  
#x;;  
#let y = "Bonjour" in y ^ " a tous.";;  
#y;;  
#x + 1;;  
#let x = 4 in x + 2;;  
#x;;
```

Vérifier vos prévisions en tapant effectivement ces lignes.

Exercice 2 : Mêmes questions avec les commandes suivantes :

```
#let x = 2;;  
#let x = 3 in  
  let y = x + 1 in  
    x + y;;  
#let x = 3 and y = x + 1 in  
  x + y;;
```

Pourquoi les deux dernières commandes ne fournissent-elles pas le même résultat ?

Exercice 3 : Expliquer le comportement suivant.

```
#let x = 3;;  
x : int = 3  
#let f y = y + x;;  
f : int -> int = <fun>  
#f 2;;  
- : int = 5  
#let x = 0;;  
x : int = 0  
#f 2;;  
- : int = 5
```

3 Premiers programmes

Créez un fichier `tp1.ml` dans votre éditeur de texte préféré. Dans chacun des exercices ci-dessous, on vous demande d'écrire une fonction OCaml, que vous placerez dans le fichier `tp1.ml`. Veuillez à bien tester chaque fonction : pour cela, ouvrez un toplevel et chargez votre fichier `.ml` avec la commande

```
#use "tp1.ml"
```

Vous pourrez ensuite tester chaque fonction sur diverses entrées. Attention : si vous modifiez le code d'une fonction, il faudra à nouveau la charger dans le toplevel pour que celui-ci se mette à jour.

Alternativement, en ouvrant le fichier `tp1.ml` sous emacs, vous pouvez aisément lancer un toplevel et lui soumettre vos définitions de fonctions et vos tests avec le raccourci `Ctrl-x`, `Ctrl-e`.

Exercice 4 : Définir une fonction `square` attendant un `int` et renvoyant son carré.

Exercice 5 : Définir une fonction `perimeter` attendant un `float` représentant le rayon d'un cercle et renvoyant le périmètre de ce cercle. On rappelle que le périmètre d'un cercle de rayon r vaut $2\pi r$ et que π vaut environ 3.1415927. Pour définir la valeur de cette constante, servez-vous d'une définition locale.

Exercice 6 : Écrire une fonction `div` attendant deux valeurs n et m de type `int` et renvoyant la division en virgule flottante de n par m . Par exemple, `div 3 4` donnera 0.75. Quel est le resultat de `div 3 0`? Et `div 0 0`? Rappel : la fonction `float_of_int` permet de convertir un entier en nombre flottant.

Exercice 7 : Définir une fonction `bis` attendant une chaîne de caractères et renvoyant cette chaîne concaténée à elle-même. Par exemple, `bis "ab"` donnera `"abab"`. L'opérateur de concaténation de chaînes de caractères s'écrit `^`.

Exercice 8 : Écrire une fonction `times8` attendant une chaîne de caractères et renvoyant huit exemplaires de cette chaîne concaténés à la suite. On notera que :

- en concaténant `"ab"` à elle-même, on obtient `"abab"` (deux fois `"ab"`)
- en concaténant `"abab"` à elle-même, on obtient `"abababab"` (quatre fois `"ab"`)
- en concaténant `"abababab"` à elle-même, on obtient `"abababababababab"` (huit fois `"ab"`)

Servez-vous de cette propriété pour construire le résultat de cette fonction à l'aide d'une suite de définitions locales.

Exercice 9 : Sauriez-vous écrire une seconde version de cette fonction nommée `times8_bis` ne se servant que de la fonction `bis` ci-dessus, sans définitions locales, et déléguant à `bis` toutes les concaténations ?

Exercice 10 : Comme indiqué ci-dessus, toutes les expressions de comparaison sont de type `bool`, donc s'évaluent en `true` ou en `false`. En déduire une fonction `is_zero` : `int -> bool` prenant un argument un entier et renvoyant `true` si cet entier est nul, `false` sinon.

Exercice 11 : Écrire une fonction `msg_zero` : `int -> string` prenant un argument un entier et renvoyant la chaîne de caractères `"zero"` si cet entier est nul, la chaîne `"not zero"` sinon.

Exercice 12 : La fonction `max`: `'a -> 'a -> 'a` est prédéfinie en OCaml : appliquée à a et b de même type, elle renvoie le maximum de a et b .

- Testez la fonction `max` prédéfinie sur différents types d'arguments : `int`, `float`, `char`, `string`, ... Essayez-la aussi sur des couples de valeurs, de types éventuellement distincts.
- En l'appelant `my_max`, donnez votre propre implémentation de la fonction `max` – écrite à l'aide d'un `if`, et sans vous servir de `max`. Votre fonction doit être de même type que le `max` prédéfini, et donner les mêmes résultats.
- À partir de la seule fonction `max` (ou `my_max`, peu importe) et sans `if`, définir :
 - une fonction `max_triple` prenant trois arguments a , b et c et renvoyant le plus grand,
 - une fonction `max_quadruple` prenant quatre arguments a , b , c et d et renvoyant le plus grand.

Exercice 13 : Rappelons que la somme des entiers de 0 à n peut être définie récursivement par $\Sigma(0) = 0$, et $\Sigma(n) = n + \Sigma(n - 1)$ si $n > 0$. Écrire cette fonction en Caml. Que donne cette fonction appliquée à un nombre négatif ?

Exercice 14 : Écrire en Caml la fonction récursive définie par $f(0) = 1$, $f(1) = 1$ et $f(n) = f(n - 1) + f(n - 2)$.

Exercice 15 : Les fonctions en Caml sont des valeurs comme les autres (c'est pour cela que l'on dit que Caml est un langage *fonctionnel*). En particulier, une fonction peut prendre en argument une ou plusieurs fonctions, et renvoyer une fonction :

```
let composer f g = (fun x -> f (g x));;
```

Écrire une fonction prenant en argument une fonction `f` et un entier `n`, et renvoyant $\sum_{i=0}^n f(i)$. Quel sera son type ? Utiliser cette fonction pour définir une nouvelle fonction calculant la somme des $n + 1$ premiers carrés.

Exercice 16 : Écrire une fonction `binome p n` renvoyant $\binom{n}{p}$, donné par :

$$\begin{aligned} \binom{n}{p} &= \binom{n-1}{p} + \binom{n-1}{p-1} && \text{si } 1 \leq p \leq n \\ \binom{n}{0} &= 1 \\ \binom{n}{p} &= 0 && \text{si } p > n. \end{aligned}$$

Exercice 17 : Les variables prédéfinies `max_int` et `min_int` valent respectivement la plus grande valeur et la plus petite valeur du type `int`. Examiner les valeurs de ces deux variables.

À l'aide des fonctions de conversion `float_of_int` et `int_of_float`, calculer la valeur de la conversion en `int` de la conversion en `float` de `max_int`. Retrouvez-vous `max_int` ?

Nous vous montrerons comment résoudre ce type de problème lorsque nous vous parlerons des modules : il existe des bibliothèques permettant de manipuler des entiers arbitrairement grands (dans les limites, bien sûr, de la mémoire de la machine).